

```
In [ ]: # 3.2 Decision Tree регрессия.
```

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
In [ ]: ## 1. Загрузка и обработка датасета.
Датасет, его загрузка и обработка как в заданиях 1.2, 2.2.
```

```
In [3]: df_air = pd.read_csv(
    'C:/Users/Elizaveta/Downloads/air_quality/AirQuality.csv',
    delimiter=';',
    decimal=',',
    na_values=[' ', 'NA', 'nan', -200]
)
print(f"размер после загрузки: {df_air.shape}")
print("столбцы после загрузки:", list(df_air.columns))
df_air = df_air.loc[:, ~df_air.columns.str.contains('Unnamed')]
print(f"\nразмер таблицы после удаления пустых столбцов: {df_air.shape}")
print("столбцы:", list(df_air.columns))
print(df_air.head())
print(df_air.info())
print(df_air.isnull().sum())
```

размер после загрузки: (9471, 17)

столбцы после загрузки: ['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH', 'AH', 'Unnamed: 15', 'Unnamed: 16']

размер таблицы после удаления пустых столбцов: (9471, 15)

столбцы: ['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH', 'AH']

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	\
0	10/03/2004	18.00.00	2.6	1360.0	150.0	11.9	
1	10/03/2004	19.00.00	2.0	1292.0	112.0	9.4	
2	10/03/2004	20.00.00	2.2	1402.0	88.0	9.0	
3	10/03/2004	21.00.00	2.2	1376.0	80.0	9.2	
4	10/03/2004	22.00.00	1.6	1272.0	51.0	6.5	

	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	\
0	1046.0	166.0	1056.0	113.0	1692.0	1268.0	
1	955.0	103.0	1174.0	92.0	1559.0	972.0	
2	939.0	131.0	1140.0	114.0	1555.0	1074.0	
3	948.0	172.0	1092.0	122.0	1584.0	1203.0	
4	836.0	131.0	1205.0	116.0	1490.0	1110.0	

	T	RH	AH
0	13.6	48.9	0.7578
1	13.3	47.7	0.7255
2	11.9	54.0	0.7502
3	11.0	60.0	0.7867
4	11.2	59.6	0.7888

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 9471 entries, 0 to 9470

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	Date	9357 non-null	object
1	Time	9357 non-null	object
2	CO(GT)	7674 non-null	float64
3	PT08.S1(CO)	8991 non-null	float64
4	NMHC(GT)	914 non-null	float64
5	C6H6(GT)	8991 non-null	float64
6	PT08.S2(NMHC)	8991 non-null	float64
7	NOx(GT)	7718 non-null	float64
8	PT08.S3(NOx)	8991 non-null	float64
9	NO2(GT)	7715 non-null	float64
10	PT08.S4(NO2)	8991 non-null	float64
11	PT08.S5(O3)	8991 non-null	float64
12	T	8991 non-null	float64
13	RH	8991 non-null	float64
14	AH	8991 non-null	float64

dtypes: float64(13), object(2)

memory usage: 1.1+ MB

None

Date 114

Time 114

CO(GT) 1797

PT08.S1(CO) 480

NMHC(GT) 8557

C6H6(GT) 480

PT08.S2(NMHC) 480

NOx(GT) 1753

```
PT08.S3(NOx)      480
NO2(GT)           1756
PT08.S4(NO2)      480
PT08.S5(O3)       480
T                 480
RH                480
AH                480
dtype: int64
```

```
In [7]: df_air = df_air.drop(['Date', 'Time'], axis=1, errors='ignore')
target_column = 'C6H6(GT)'
column_to_drop = ['NMHC(GT)', 'Hour', 'DayOfWeek', 'Month', target_column]
numeric_columns = df_air.select_dtypes(include=[np.number]).columns
feature_columns = [columns for columns in numeric_columns if columns not in column_to_drop]
print(f"\nИспользуемые признаки:")
for i, columns in enumerate(feature_columns, 1):
    print(f"{i}. {columns}")
```

Используемые признаки:

1. CO(GT)
2. PT08.S1(CO)
3. PT08.S2(NMHC)
4. NOx(GT)
5. PT08.S3(NOx)
6. NO2(GT)
7. PT08.S4(NO2)
8. PT08.S5(O3)
9. T
10. RH
11. AH

```
In [8]: X = df_air[feature_columns]
Y = df_air[target_column]
mask = X.notnull().all(axis=1)&Y.notnull()
X_clean = X[mask]
Y_clean = Y[mask]
print(f"Размер X: {X_clean.shape}")
print(f"Размер Y: {Y_clean.shape}")
print(f"\nЗначения целевой переменной {target_column}:")
print(f"Min: {Y_clean.min():.3f}")
print(f"Max: {Y_clean.max():.3f}")
print(f"Среднее: {Y_clean.mean():.3f}")
print(f"Медиана: {Y_clean.median():.3f}")
print(f"Стандартное отклонение: {Y_clean.std():.3f}")
```

Размер X: (6941, 11)

Размер Y: (6941,)

Значения целевой переменной C6H6(GT):

Min: 0.200

Max: 63.700

Среднее: 10.554

Медиана: 8.800

Стандартное отклонение: 7.465

```
In [9]: from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X_clean, Y_clean,
                                                    test_size=0.3,
                                                    random_state=42)

print(f"Обучающая выборка: {X_train.shape}")
print(f"Тестовая выборка: {X_test.shape}")
```

Обучающая выборка: (4858, 11)

Тестовая выборка: (2083, 11)

```
In [11]: ##2. Создание бейзлайна.
Создаём базовую модель с использованием DecisionTreeRegressor из sklearn.
Обучающая выборка (train) используется для обучения модели.
Тестовая выборка (test) используется для оценки качества обученной модели.
Используются метрики MAE, RMSE, R2.
MAE=0.0214 показывает среднюю величину ошибки в тех же единицах, что и целевая п
RMSE=0.1463 также показывает среднюю ошибку переменной, но за счёт возведения в
демонстрирует наличие выбросов.
R2=0.9985 показывает долю дисперсии целевой переменной, полученное значение подт
Базовая модель демонстрирует возможное переобучение, типичное для деревьев. Она
```

Cell In[11], line 2

Создаём базовую модель с использованием DecisionTreeRegressor из sklearn.

^

SyntaxError: invalid syntax

```
In [14]: from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
DTree_Regressor = DecisionTreeRegressor(random_state=42)
DTree_Regressor.fit(X_train, Y_train)
Y_prediction = DTree_Regressor.predict(X_test)
MAE = mean_absolute_error(Y_test, Y_prediction)
RMSE = np.sqrt(mean_squared_error(Y_test, Y_prediction))
R2 = r2_score(Y_test, Y_prediction)
print(f"MAE: {MAE:.4f}")
print(f"RMSE: {RMSE:.4f}")
print(f"R2: {R2:.4f}")
```

MAE: 0.0214

RMSE: 0.2944

R2: 0.9985

```
In [ ]: ##3. Улучшение бейзлайна.
Гипотеза 1: Подбор гиперпараметров.
После построения базовой модели, показывающей аномально высокие значения метрик,
ограничивающие сложность дерева. Для этого используется перебор возможных комбин
минимальное число объектов, необходимое для разделения узла, минимальное число с
при максимизации получить минимальную среднеквадратичную ошибку. Выполняется крос
Результат:
```

```
In [20]: from scipy.stats import randint
from sklearn.model_selection import RandomizedSearchCV
param_dist_reg = {
    'max_depth': [3, 5, 7, 10, 15, None],
    'min_samples_split': randint(2, 20),
    'min_samples_leaf': randint(1, 10),
    'criterion': ['squared_error', 'absolute_error', 'friedman_mse']
}

random_reg = RandomizedSearchCV(
```

```
DecisionTreeRegressor(random_state=42),
param_distributions=param_dist_reg,
n_iter=50,
cv=5,
scoring='neg_mean_squared_error',
random_state=42,
n_jobs=-1
)
random_reg.fit(X_train, Y_train)
print("Лучшие параметры", random_reg.best_params_)
```

Лучшие параметры {'criterion': 'squared_error', 'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_split': 2}

In []: Тестирование на тестовой выборке улучшений метрик не дало. MAE = 0.0206, RMSE =

```
In [21]: best_tree_reg = DecisionTreeRegressor(
        criterion='squared_error',
        max_depth=10,
        min_samples_leaf=2,
        min_samples_split=2,
        random_state=42
    )
best_tree_reg.fit(X_train, Y_train)
Y_pred_reg = best_tree_reg.predict(X_test)
mae_reg = mean_absolute_error(Y_test, Y_pred_reg)
rmse_reg = np.sqrt(mean_squared_error(Y_test, Y_pred_reg))
r2_reg = r2_score(Y_test, Y_pred_reg)
print(f"Улучшенное дерево: MAE = {mae_reg:.4f}, RMSE = {rmse_reg:.4f}, R2 = {r2_
```

Улучшенное дерево: MAE = 0.0206, RMSE = 0.2938, R2 = 0.9985

In []: Тестирование на обучающей выборке с высокой вероятностью указывает на переобучен

```
In [13]: best_tree_reg = DecisionTreeRegressor(
        criterion='squared_error',
        max_depth=10,
        min_samples_leaf=2,
        min_samples_split=2,
        random_state=42
    )
best_tree_reg.fit(X_train, Y_train)
Y_train_pred = best_tree_reg.predict(X_train)
mae_train = mean_absolute_error(Y_train, Y_train_pred)
rmse_train = np.sqrt(mean_squared_error(Y_train, Y_train_pred))
r2_train = r2_score(Y_train, Y_train_pred)
print(f"Train: MAE={mae_train:.4f}, RMSE={rmse_train:.4f}, R2={r2_train:.4f}")
```

Train: MAE=0.0046, RMSE=0.0233, R2=1.0000

In []: Попробуем подобрать гиперпараметры с более жестким ограничением. увеличены миним
Новая оптимальная комбинация: {'criterion': 'squared_error', 'max_depth': 7, 'mi
Test: MAE = 0.0750, RMSE = 0.3190, R2 = 0.9982
Train: MAE=0.0643, RMSE=0.1035, R2=0.9998
Модель обладает немного более качественной обобщающей способностью с этими парам
низким уровнем шума в данных.

```
In [23]: param_dist_reg = {
        'max_depth': [3, 5, 7, 10],
        'min_samples_split': randint(5, 20),
```

```

    'min_samples_leaf': randint(5, 20),
    'criterion': ['squared_error', 'absolute_error']
}
random_reg = RandomizedSearchCV(
    DecisionTreeRegressor(random_state=42),
    param_distributions=param_dist_reg,
    n_iter=50,
    cv=5,
    scoring='neg_mean_squared_error',
    random_state=42,
    n_jobs=-1
)
random_reg.fit(X_train, Y_train)
print("Лучшие параметры:", random_reg.best_params_)
best_tree_reg2 = random_reg.best_estimator_
Y_pred = best_tree_reg2.predict(X_test)
mae = mean_absolute_error(Y_test, Y_pred)
rmse = np.sqrt(mean_squared_error(Y_test, Y_pred))
r2 = r2_score(Y_test, Y_pred)
print(f"MAE = {mae:.4f}, RMSE = {rmse:.4f}, R2 = {r2:.4f}")
best_tree_reg2.fit(X_train, Y_train)
Y_train_pred = best_tree_reg2.predict(X_train)
mae_train = mean_absolute_error(Y_train, Y_train_pred)
rmse_train = np.sqrt(mean_squared_error(Y_train, Y_train_pred))
r2_train = r2_score(Y_train, Y_train_pred)
print(f"Train: MAE={mae_train:.4f}, RMSE={rmse_train:.4f}, R2={r2_train:.4f}")

```

Лучшие параметры: {'criterion': 'squared_error', 'max_depth': 7, 'min_samples_leaf': 5, 'min_samples_split': 7}

MAE = 0.0750, RMSE = 0.3190, R2 = 0.9982

Train: MAE=0.0643, RMSE=0.1035, R2=0.9998

In []: Гипотеза 2: Обрезка решающего дерева.
 Обрезка решающего дерева для улучшения обобщающей способности. Этот метод уменьшает сложность модели, что помогает бороться с переобучением и улучшает качество модели. Для каждого возможного значения α строится соответствующее дерево и вычисляется массив альфа-значений. Для каждого десятого текущего ccp_alpha в модели и оценка качества.
 Результаты:
 Лучший ccp_alpha : 0.000005
 MAE = 0.0243, RMSE = 0.2962, R2 = 0.9985
 Показатели практически идентичны полученным после первого подбора гиперпараметров оптимальному даже без ограничений, а обрезка не даёт дополнительный выигрыш.

```

In [15]: from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import cross_val_score
tree = DecisionTreeRegressor(random_state=42)
tree.fit(X_train, Y_train)
path = tree.cost_complexity_pruning_path(X_train, Y_train)
ccp_alphas = path.ccp_alphas
ccp_alphas = np.unique(ccp_alphas)[::-1]
rmse_scores = []
for alpha in ccp_alphas:
    tree_pruned = DecisionTreeRegressor(random_state=42, ccp_alpha=alpha)
    scores = -cross_val_score(tree_pruned, X_train, Y_train, cv=5, scoring='neg_mean_squared_error')
    rmse_scores.append(np.sqrt(scores.mean()))
best_alpha = ccp_alphas[np.argmin(rmse_scores)]
print(f"Лучший ccp_alpha: {best_alpha:.6f}")
best_tree = DecisionTreeRegressor(random_state=42, ccp_alpha=best_alpha)
best_tree.fit(X_train, Y_train)
Y_pred = best_tree.predict(X_test)

```

```

mae = mean_absolute_error(Y_test, Y_pred)
rmse = np.sqrt(mean_squared_error(Y_test, Y_pred))
r2 = r2_score(Y_test, Y_pred)
print(f"MAE = {mae:.4f}, RMSE = {rmse:.4f}, R2 = {r2:.4f}")

```

Лучший csp_alpha: 0.000005

MAE = 0.0243, RMSE = 0.2962, R2 = 0.9985

In []: *## 4. Имплементация алгоритма машинного обучения.*
 Был создан собственный алгоритм решающего дерева MyDecisionTreeRegressor. Состоит (регрессор, содержит методы). Основные методы:
 __init__ - задание гиперпараметров.
 fit - обработка входных данных, запуск построения рекурсивного дерева.
 _build_tree - рекурсивно строит дерево.
 _best_split - ищет лучшее разбиение.
 _impurity - вычисляет меру неопределённости для массива меток.
 predict - формирование предсказаний.

```

In [24]: class Node:
    def __init__(self, feature=None, threshold=None, left=None, right=None, proba=None, value=None):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.proba = proba
        self.value = value

    class MyDecisionTreeRegressor:
        def __init__(self, max_depth=None, min_samples_split=2, min_samples_leaf=1):
            self.max_depth = max_depth
            self.min_samples_split = min_samples_split
            self.min_samples_leaf = min_samples_leaf
            self.root = None
            self.n_features_ = None

        def fit(self, X, y):
            X = np.array(X)
            y = np.array(y)
            self.n_features_ = X.shape[1]
            self.root = self._build_tree(X, y, depth=0)
            return self

        def _build_tree(self, X, y, depth):
            n_samples, n_features = X.shape
            var_y = np.var(y)
            if (depth == self.max_depth) or (n_samples < self.min_samples_split) or (n_samples < self.min_samples_leaf):
                leaf_value = np.mean(y)
                return Node(value=leaf_value)
            best_feat, best_thresh, best_gain = self._best_split(X, y)
            if best_feat is None:
                leaf_value = np.mean(y)
                return Node(value=leaf_value)
            left_mask = X[:, best_feat] <= best_thresh
            right_mask = ~left_mask
            left_child = self._build_tree(X[left_mask], y[left_mask], depth + 1)
            right_child = self._build_tree(X[right_mask], y[right_mask], depth + 1)
            return Node(feature=best_feat, threshold=best_thresh, left=left_child, right=right_child, value=None)

        def _best_split(self, X, y):
            best_gain = -float('inf')
            best_feature = None
            best_threshold = None
            current_var = np.var(y)
            for feature in range(self.n_features_):

```

```

thresholds = np.unique(X[:, feature])
for thresh in thresholds:
    left_mask = X[:, feature] <= thresh
    right_mask = ~left_mask
    if np.sum(left_mask) < self.min_samples_leaf or np.sum(right_mas
        continue
    left_var = np.var(y[left_mask])
    right_var = np.var(y[right_mask])
    n_left = np.sum(left_mask)
    n_right = np.sum(right_mask)
    n_total = n_left + n_right
    gain = current_var - (n_left / n_total) * left_var - (n_right /
    if gain > best_gain:
        best_gain = gain
        best_feature = feature
        best_threshold = thresh
if best_gain <= 0:
    return None, None, None
return best_feature, best_threshold, best_gain
def predict(self, X):
    X = np.array(X)
    predictions = []
    for x in X:
        node = self.root
        while node.value is None:
            if x[node.feature] <= node.threshold:
                node = node.left
            else:
                node = node.right
        predictions.append(node.value)
    return np.array(predictions)

```

In []: Проверка работы собственного решающего дерева. Результаты:
 MAE = 0.2668, RMSE = 0.4509, R2 = 0.9964
 MAE и RMSE значительно выше, чем на библиотечной модели (MAE: 0.0214, RMSE: 0.29
 работы алгоритма.

In [26]: my_tree_reg = MyDecisionTreeRegressor(max_depth=5, min_samples_split=5, min_samp
 my_tree_reg.fit(X_train, Y_train)
 Y_pred_reg = my_tree_reg.predict(X_test)
 mae = mean_absolute_error(Y_test, Y_pred_reg)
 rmse = np.sqrt(mean_squared_error(Y_test, Y_pred_reg))
 r2 = r2_score(Y_test, Y_pred_reg)
 print(f"Моё дерево : MAE = {mae:.4f}, RMSE = {rmse:.4f}, R2 = {r2:.4f}")

Моё дерево : MAE = 0.2668, RMSE = 0.4509, R2 = 0.9964

In []: Проверка работы собственного алгоритма с подобранными оптимальными параметрами.
 MAE = 0.0752
 RMSE = 0.3195
 R2 = 0.9982
 Библиотечная модель с теми же параметрами показала идентичные результаты, что по

In [27]: my_tree_reg_optimized = MyDecisionTreeRegressor(
 max_depth=7,
 min_samples_split=7,
 min_samples_leaf=5
)
 my_tree_reg_optimized.fit(X_train, Y_train)


```
Y_pred_optimized = my_tree_reg_optimized.predict(X_test)
mae_opt = mean_absolute_error(Y_test, Y_pred_optimized)
rmse_opt = np.sqrt(mean_squared_error(Y_test, Y_pred_optimized))
r2_opt = r2_score(Y_test, Y_pred_optimized)
print("Моя реализация (оптимальные параметры):")
print(f"MAE = {mae_opt:.4f}")
print(f"RMSE = {rmse_opt:.4f}")
print(f"R2 = {r2_opt:.4f}")
```

Моя реализация (оптимальные параметры):

MAE = 0.0752

RMSE = 0.3195

R2 = 0.9982

In []: *## Выводы:*

В ходе исследования решающего дерева регрессии была построена базовая модель из гиперпараметров она показала на тренировочной выборке идеальные метрики, указывая. Для адекватной работы модели был проведён еще один подбор гиперпараметров с помощью. Применение обрезки дерева с подбором `ccp_alpha` показало оптимальное значение `0.0`. Исходные данные обладают сильной структурой и даже глубокие деревья не должны быть способны лучше использовать модель с ограничением глубины и минимальных листьев. Показатели, полученные при обучении со стандартными (`MAE = 0.2668`, `RMSE = 0.4509`) параметрами подтвердили его корректность.